
Below is a concise review of **important notes** for the **Programming in C** course (C-23, CM-106), tailored for quick revision based on the C-23 syllabus (DCME C-23 SYLLUBUS-58-63.pdf) and the provided exam paper (C23-CM-106 23047 March/April 2024). These notes focus on high-yield topics, key concepts, and frequently tested areas, organized by chapter for efficient study.

Quick Review: Important Notes for Programming in C (C-23, CM-106)

1. Introduction to C Language

History: Developed by Dennis Ritchie, 1972, at Bell Labs; basis for modern languages.

Program Structure: Preprocessor directives (`#include`), main function, declarations, statements.

C Tokens: Keywords (e.g., `int`, `if`), identifiers (variable names), constants (e.g., 5), operators (e.g., `+`), special symbols (e.g., `;`).

Data Types:

Basic: `int` (e.g., 10), `float` (e.g., 3.14), `char` (e.g., 'A'), `double`.

Derived: Arrays, structures, pointers.

User-defined: `typedef`, `enum`.

Variables/Constants:

Variable: Declared (e.g., `int x;`), initialized (e.g., `int x = 5;`).

Constant: Fixed value (e.g., `const int MAX = 100;`).

Type Qualifiers: `const`, `volatile`, `restrict` for variable behavior.

Execution Steps: Edit (write code), compile (generate object code), execute (run), debug (fix errors).

Exam Tip: Memorize data types, tokens, and program structure (Q1, Q2, Q11 frequent).

2. Input/Output, Operators, and Expressions

Preprocessor: `#include <stdio.h>` for I/O functions.

I/O Functions:

Input: `getchar()` (single char), `getche()` (echoed char), `gets()` (string), `scanf()` (formatted, e.g., `scanf("%d", &x);`).

Output: `putchar()` (single char), `puts()` (string), `printf()` (formatted, e.g., `printf("%d", x);`).

Operators:

Arithmetic: `+`, `-`, `*`, `/`, `%` (e.g., `5 % 2 = 1`).

Relational: `==`, `!=`, `<`, `>`, `<=`, `>=` (e.g., `x > y`).

Logical: `&&`, `||`, `!` (e.g., `(x > 0 && y < 10)`).

Bitwise: `&`, `|`, `^`, `~`, `<<`, `>>` (e.g., `5 & 3 = 1`).

Assignment: `=`, `+=`, `-=`, etc. (e.g., `x += 5`).

Increment/Decrement: `++`, `--` (e.g., `x++`).

Conditional: `?:` (e.g., `max = (a > b) ? a : b;`).

Special: Comma `,`, `sizeof` (e.g., `sizeof(int)`).

Precedence: `()` > `++` > `*` > `+` > `&&` > `||` (use parentheses for clarity).

Type Conversion: Implicit (e.g., `int` to `float`) or explicit (e.g., `(float)x`).

Exam Tip: Focus on operator precedence, `scanf` / `printf` syntax, logical operators (Q3, Q4, Q12).

3. Decision Making, Iterative, and Control Statements

Decision Making:

if: `if (condition) { statements; }.`

if-else: `if (condition) { statements; } else { statements; }.`

Nested if-else: Multiple `if` within `if`.

if-else-if Ladder: `if (cond1) {} else if (cond2) {} else {}.`

switch: `switch (expression) { case value: statements; break; }.`

Loops:

while: `while (condition) { statements; }.`

do-while: `do { statements; } while (condition);` (runs at least once).

for: `for (init; condition; update) { statements; }.`

Nested Loops: Loop inside loop (e.g., matrix printing).

Control Statements:

break: Exits loop/switch.

continue: Skips to next iteration.

goto: Jumps to labeled statement (avoid overuse).

Structured Programming: Modular, sequential code for clarity.

Exam Tip: Practice programs for prime number, leap year, sum of digits; know

`switch` vs. `if-else` (Q5, Q6, Q13, Q14).

4. Arrays, Strings, Structures, and Unions

Arrays:

1D Array: Declare (`int arr[5];`), initialize (`int arr[] = {1, 2, 3};`), access (`arr[0]`).

2D Array: Declare (`int matrix[2][3];`), initialize, use for matrix operations (addition, multiplication).

Programs: Sorting, largest element, matrix addition.

Strings:

Definition: Array of chars (e.g., `char str[] = "hello";`).

Functions: `gets()` (input), `puts()` (output), `strlen()`, `strcpy()`, `strcat()`, `strcmp()` (string.h).

Character Arithmetic: `'A' + 1 = 'B'` .

Structures:

Definition: `struct Student { int id; char name[20]; };` .

Access: Dot operator (e.g., `student.id`).

Features: Nested structures, array of structures, structure assignment.

Unions:

Definition: `union Data { int i; float f; };` (shares memory).

Difference: Structure stores all members; union stores one at a time.

Exam Tip: Practice matrix programs, string functions, structure declarations (Q7, Q8, Q15, Q16).

5. User-Defined Functions, Pointers, File Management, Preprocessors

Functions:

Definition: `return_type function_name(parameters) { body; }`.

Types:

No arguments, no return (e.g., `void print() {}`).

Arguments, no return (e.g., `void sum(int a, int b) {}`).

Arguments, return (e.g., `int add(int a, int b) { return a+b; }`).

No arguments, return (e.g., `int getValue() { return 5; }`).

Recursion: Function calls itself (e.g., factorial).

Storage Classes: `auto` (local), `static` (retains value), `extern` (global), `register` (fast access).

Pointers:

Definition: Stores address (e.g., `int *ptr;`).

Operators: `&` (address), `*` (dereference).

Uses: Array access (`*(ptr+i)`), function arguments, dynamic memory (`malloc`, `free`).

Features: Pointer arithmetic, pointer arrays, structures with pointers.

File Handling:

File Pointer: `FILE *fp;`

Modes: `r` (read), `w` (write), `a` (append).

Functions: `fopen()`, `fclose()`, `fscanf()`, `fprintf()`, `fgets()`, `fputs()`.

Random Access: `fseek()`, `ftell()`.

Preprocessor Directives:

Examples: `#include`, `#define MAX 100`, `#ifdef`.

Uses: Macros, conditional compilation.

Command Line Arguments: `int main(int argc, char *argv[])`.

Exam Tip: Practice factorial function, file I/O programs, pointer declarations (Q9, Q10, Q17, Q18).

Study Tips:

Prioritize: Program structure (Q11), operator precedence (Q12), loop programs (Q13), string/array programs (Q14, Q15), structures/unions (Q16), functions/files (Q17, Q18).

Memorize: Data types, operator precedence, loop/switch syntax, string functions, file modes.

Practice: Write programs for prime number, matrix addition, string reverse, factorial, file I/O.

Marks Distribution: Syllabus weightage—Introduction (16), Operators (16), Control Statements (26), Arrays/Structures (26), Functions/Files (26)—guides question focus.

Exam Pattern: Part-A (10x3=30, short answers), Part-B (5x10=50, programs/theory), based on 2024 paper.

Notes:

Focus: High-yield, concise points covering all chapters, emphasizing exam-tested topics (e.g., data types, loops, arrays, functions, files).

Source: Derived from C-23 CM-106 syllabus and March/April 2024 exam for relevance.

Format: Bullet points for quick review, tailored for last-minute prep.

Assumption: No syllabus changes for April 2025.

If you need specific sections expanded, practice programs, or a different format, let me know!